

Security Fundamentals

Lecture 7

laksith

Security Basics

Why do we care?

- Basic government functions at risk: Voting security sucks
- Hospitals suffering from ransomware attacks (DNI)
- Financial security
  - Equifax breach, First American Financial
- Personal information
- Job security!
  - According to the Bureau of Labor Statistics, 33% predicted increase in jobs from 2022 to 2032 (most hover at 3%)

Basic Principles

- Security is economics
- Least privilege
- Defense in depth
- Complete mediation
- Accounting for human factors

Most important: **know your threat model**

Understand what is at risk and what you can do to minimize risk

Security Goals

1. Confidentiality
    a. Ensure only those with approved access can read data
2. Integrity
    a. Ensure data has not been tampered with
3. Authentication
    a. Prove the author/source of data
4. Availability
    a. Ensure the uptime of a service

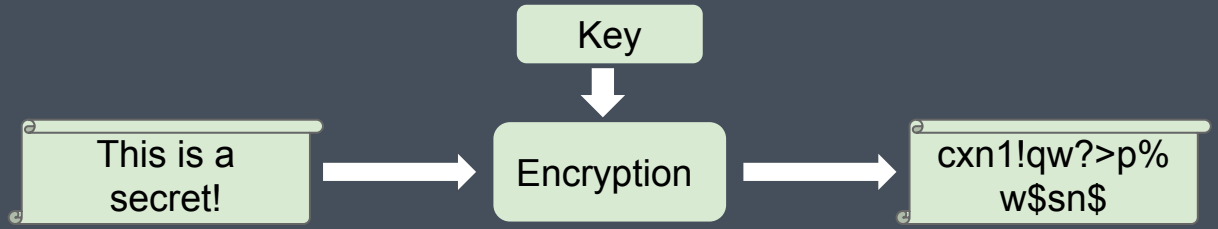
1. Confidentiality

Ensure only those with approved access can read data

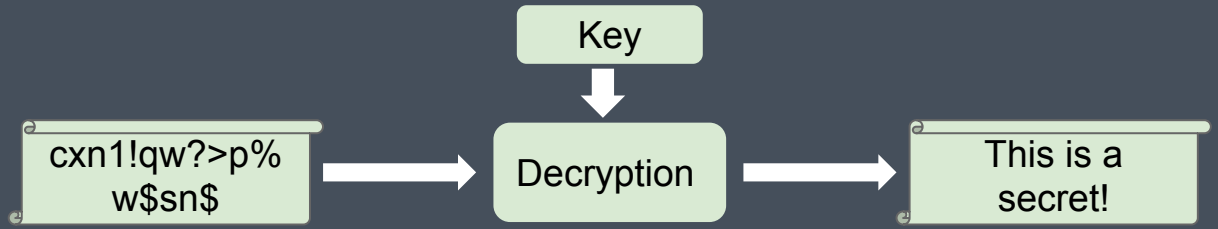
- Plaintext:
  - Vulnerable data
  - What you want to hide from the attacker
- Ciphertext:
  - Secured data that is indistinguishable from garble
  - What you want the attacker to see
- Key:
  - Secret necessary for converting plaintext into ciphertext and vice-versa

1. Confidentiality

- Encryption: plaintext + key → ciphertext



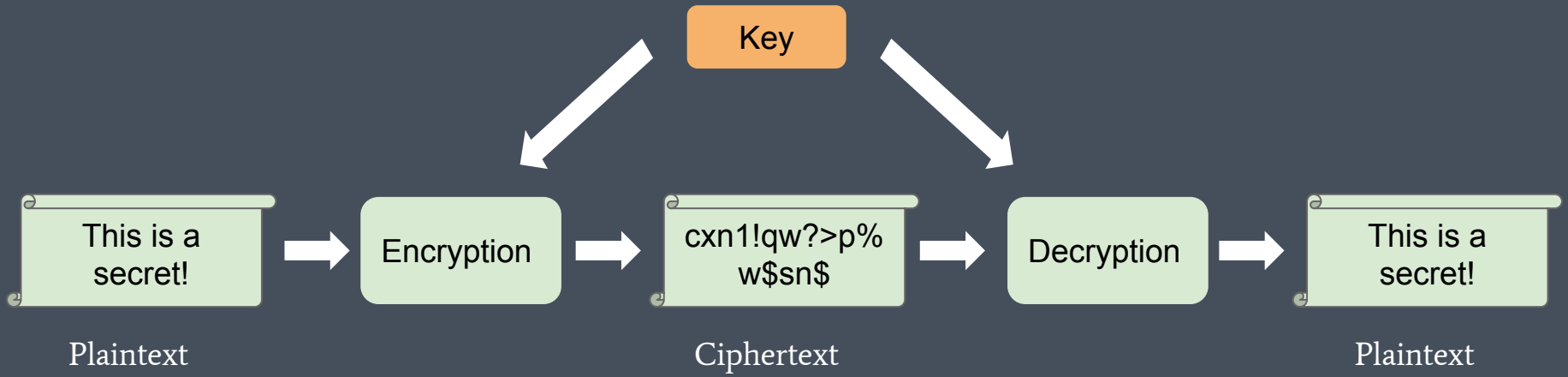
- Decryption: ciphertext + key → plaintext



1. Confidentiality

Symmetric cryptography:

Same key for encrypting and decrypting data



1. Confidentiality

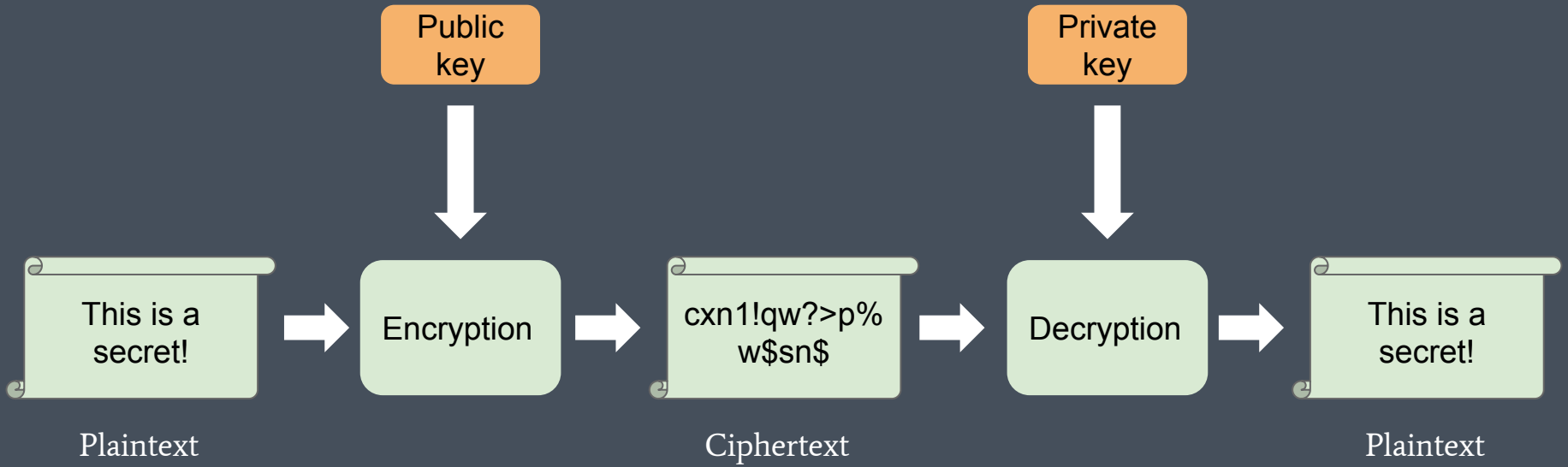
Asymmetric cryptography (AKA public key cryptography):

Comes in public-private key pairs where public key is for encryption and private key is for decryption

- Public key: can be distributed to everyone
- Private key: must be kept secret
- Anyone can encrypt data with public key but only the person possessing private key can decrypt data

1. Confidentiality

Asymmetric cryptography



2. Integrity

Ensure data has not been tampered with

- Hash function: maps arbitrary-length data to a fixed-length string of bits (known as a hash)
  - Hashes act as “summaries” of the input data



2. Integrity

Cryptographic hash functions possess properties that make it difficult to find two inputs with the same hash

- Hash-based MACs (Message Authentication Code):
  - Tag message with its hash
  - The recipient can verify whether the message was modified by re-computing the hash and comparing it with the one they received
- Checksums:
  - When a file is downloaded, its hash can be computed and checked against a reference hash. No need to compare bit by bit.

2. Integrity

It is difficult to revert a hash to its input

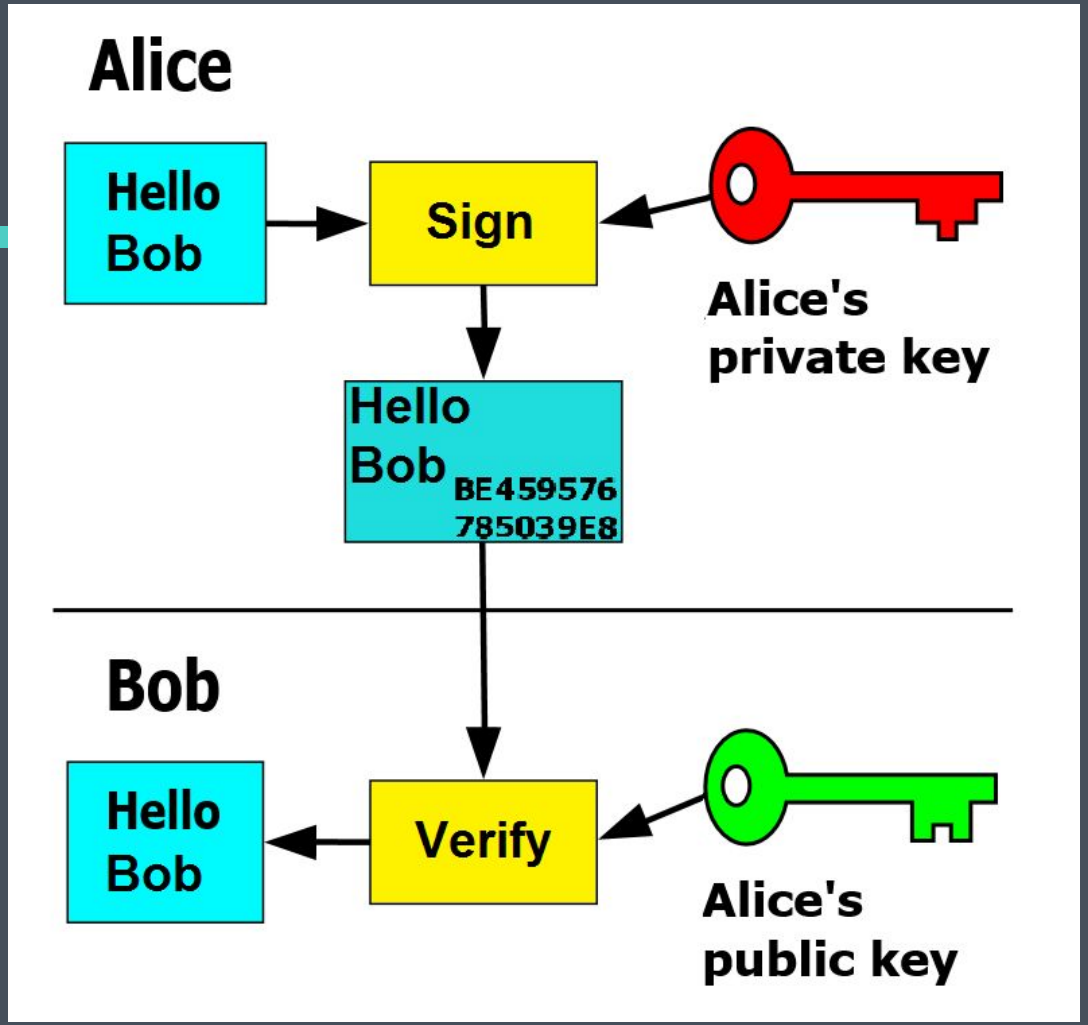
- Password storage: store hashes of passwords instead of plaintext, so in case of server breach, only hashes would be exposed (passwords cannot be recovered from hashes)

- Verifying Downloads: use terminal commands (sha256sum) to ensure the integrity of downloads (such as your favorite distribution's iso!)

3. Authentication

Prove the author/source of data

- Asymmetric cryptography (AKA public key cryptography)
- Signature: use private key to sign a file such that anyone with the public key can verify the source of the file
  - Since private key must be kept secret, only the party in possession of private key could have signed the data
  - file + private key → signature
  - signature + public key → verification


Alice
Hello Bob
Sign
Alice's private key
Hello Bob
BE459576
785039E8
Bob
Hello Bob
Verify
Alice's public key

4. Availability

Ensure systems and data are available to authorized users when they need it. Mostly applicable to services hosted on servers.

- Filtering: prevent malicious requests from reaching server.
- Load balancers: improve distribution of workloads across multiple resources.
- Redundancies: account for when a component in the system fails.
- Backups: when system goes down, bring it back up to latest state

Questions?



Security Fundamentals

File Security: Permissions and Ownership

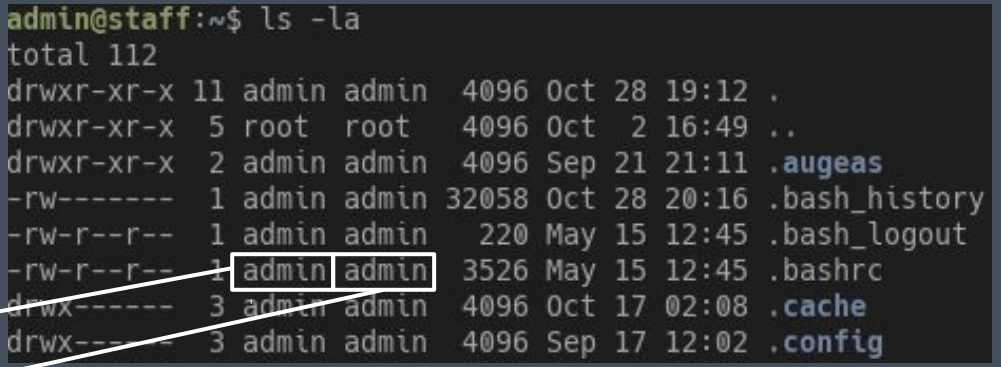
Background

- UNIX is a multi-user environment
- If multiple people can login but you have files you want to keep private (e.g., your private keys), you need a permissions and ownership setup to let you and only you access those files

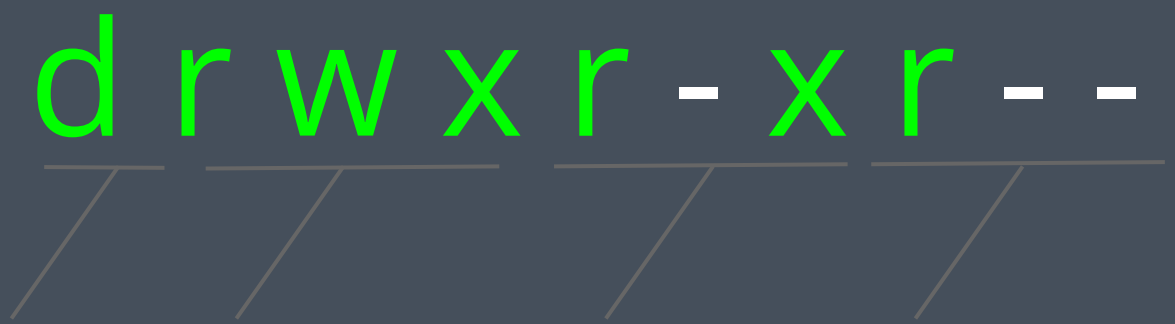


UNIX Permissions Model

- Each file has 3 “ownerships”:
  - owning user
  - owning group
  - others (everyone else)



Permissions



Modifying Permissions

2 primary ways to modify permissions/file access:

- Change file ownership: **chown**
- Change file permissions directly: **chmod**

Changing File Ownership

[sudo] chown [-R] newuser:newgroup

```
admin@staff:~/test/chown$ ls -la
total 12
drwxr-xr-x 2 admin admin 4096 Oct 31 16:49 .
drwxr-xr-x 4 admin admin 4096 Oct 31 16:49 ..
-rw-r----- 1 root  root    20 Oct 31 16:49 important_document.txt
admin@staff:~/test/chown$ cat important_document.txt
cat: important_document.txt: Permission denied
admin@staff:~/test/chown$ sudo chown admin:admin important_document.txt
admin@staff:~/test/chown$ cat important_document.txt
some important text
admin@staff:~/test/chown$ ls -la
total 12
drwxr-xr-x 2 admin admin 4096 Oct 31 16:49 .
drwxr-xr-x 4 admin admin 4096 Oct 31 16:49 ..
-rw-r----- 1 admin admin   20 Oct 31 16:49 important_document.txt
```

Changing File Permissions

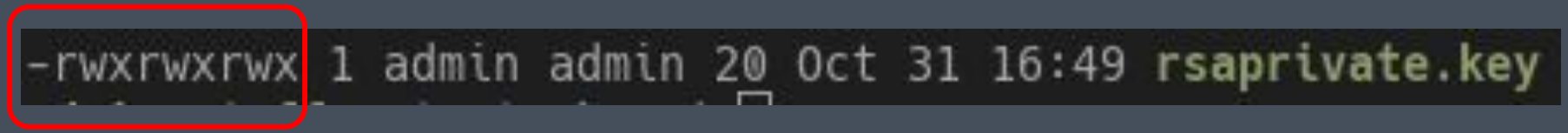
[sudo] chmod [-R] [permissions]

```
admin@staff:~/test/chown$ ls -la
total 12
drwxr-xr-x 2 admin admin 4096 Oct 31 16:49 .
drwxr-xr-x 4 admin admin 4096 Oct 31 16:49 ..
-rw-r----- 1 root  root    20 Oct 31 16:49 important_document.txt
admin@staff:~/test/chown$ cat important_document.txt
cat: important_document.txt: Permission denied
admin@staff:~/test/chown$ sudo chmod o+r important_document.txt
admin@staff:~/test/chown$ ls -la
total 12
drwxr-xr-x 2 admin admin 4096 Oct 31 16:49 .
drwxr-xr-x 4 admin admin 4096 Oct 31 16:49 ..
-rw-r--r-- 1 root  root    20 Oct 31 16:49 important_document.txt
admin@staff:~/test/chown$ cat important_document.txt
some important text
```

Why is this important?

Poor file security is one of the easiest ways to leak information or give an attacker too much privilege on your system.

What happens if you set these permissions on your private key?



Signatures and Certificates

- You can use signatures to prove the source of some data.
- But how can you prove the signer is trustworthy?
- If I gave you a public key and told you that it belonged to your bank, how do you verify this?
- This is a nontrivial problem. You have to consider things like revocation and mistakes in transmission.

Questions?



Signatures and Certificates, cont.

Enter certificates:

- A certificate is a cryptographically-signed message indicating trust in a public key
- Who signs the certificate? (i.e. who do we trust to tell us someone else is trustworthy?)
  - Luckily it's not turtles all the way down

Signatures and Certificates, cont.

- Root certificates: OS' include a number of root certificates that are the basis of trust over the network. Certificates are signed in a chain leading up to a root; if the chain is valid, the cert at the end is presumed to be trusted.
- Not a foolproof system (*cough* wosign/diginotar *cough*)

BeyondCorp and Zero Trust

- The traditional firewall + privileged intranet model doesn't really work nowadays: remote workers, PaaS, networked apps
- Google's done some work talking about a new approach with dynamic policies and more fine-grained access controls.
- Identifying access by user and device info and other heuristics
- Yes, this shit is buzzwordy as hell

Any Questions?